

Security by Design & Secure Defaults

Retrofit is expensive

Fixing security after launch costs 10-100x more than baking it in.
"Shift-left" means considering security at design, not after incidents.

The principles

Least privilege – components get the minimum rights to work

Fail-secure – errors must default to denying

Separation of duties – no single person/component holds all power

Economy of mechanism – simpler = fewer bugs

Defense in depth

Complete mediation – every access checked (not cached forever)

Secure defaults – insecure only by explicit choice

Open design – security not via secrecy of the mechanism (Kerckhoffs)

Threat modeling at design time

For every design doc: identify assets, draw data flows, apply STRIDE, rank risks, record mitigations. Architecture reviews exist to catch design-level flaws that code review cannot.

Architecture review checklist (abbreviated)

Authentication and authorization clearly scoped per component

Secrets never in code/builds; injected via vault at runtime

Input validation and output encoding at every trust boundary

Logging of security-relevant events

Session management and CSRF defense on stateful webflows

Data classification and retention defined

Encryption for data at rest and in transit

Availability: backup + recovery tested, rollback path exists

Coding defaults that matter

Frames, HSTS, CSP headers set by default in the framework

Parameterized queries default

Least-privilege DB accounts default

Error messages do not leak internals

When a framework ships an insecure default (e.g., no CORS config, debug mode on), and you forget to change it – that's a design-level failure your review checklist should have caught.

[← Zero Trust: Never Trust, Alw...](#)

[Threat Modeling in Practice: S...](#)